SOFTWARE ENGINEERING · ASSIGNMENT #04

AI-assisted evolution of the Smart Campus Room Availability Checker

Refactoring, room-catalog feature extension and automated quality evidence

Full name	**Yeraliyeva Anel**
Student ID	**24B031020**
Practice lessons	**Tuesday, 18:00 – 19:00**
AI assistant used	**Claude (Anthropic), model Claude Opus 5.5**
Date	8 November 2026

STARTING POINT AND SCOPE

Baseline: one room, one file

The Assignment #03 prototype was reconstructed as an equivalent minimal version and committed first (`48bd593`) so every later change has a recorded baseline.

- `baseline/app.js` — 94 lines: room data, rules and DOM handlers together
- One room (*Room 301*), two fictional bookings
- Fixed current time **2026-11-10 09:00**
- No login, payments, live APIs or database
- Baseline AC regression suite: **8 / 8 passed**

ID	Acceptance criterion (preserved)
AC1	Valid HH:MM, end after start, not in the past
AC2	Max 120 minutes (120 ok, 121 rejected)
AC3	Blocked room cannot be booked
AC4	Overlap rejected, touching intervals allowed
AC5	Booking data is never mutated
AC6	Clear, responsive feedback

TASK 01 · CODEBASE ANALYSIS

Structured context given to the AI assistant

Repository map (as provided)

```
baseline/
  index.html     form: date, start, end
  app.js         room data, NOW, checkAvailability(),
                 submit + change handlers
  tests/         AC1–AC5 regression checks
```

Constraints stated explicitly

- Fictional data only; no login / payments / APIs / DB
- Do not change AC1–AC6 behaviour
- Small plan first, one change at a time — no full rewrite

Requirements stated explicitly

- AC1–AC6 (previous slide), with boundary values
- New: ≥ 4 rooms (id, name, capacity, equipment, blocked, bookings)
- F1 capacity ≥ attendees · F2 ALL equipment · F3 blocked never available · F4 empty state · F5 room switch refreshes result
- ≥ 10 executable tests incl. 120/121 min, touching/overlap, exact/insufficient capacity, one UI check

Source of the context

The assignment brief (Lesson 08 slides) and the submission instructions were given to the assistant verbatim, together with the baseline file.

TASK 01 · CODEBASE ANALYSIS

Maintainability risks found in the baseline

#	Risk	Evidence (file / function / line)	Why it matters for this change
R1	**Hard-wired single room**	`app.js:6` global `room`; `checkAvailability()` line 16 `r = r || room`	Silent fallback to Room 301 — exactly the "checked another room's data" bug the new feature must avoid
R2	**Duplicated time validation**	Same HH:MM regex in `checkAvailability()` l.18, submit handler l.65, change handler l.80	Three places to update; message text also copied 2×
R3	**Duplicated result rendering**	`out.className = res.ok ? …` at l.71 and l.86	New states (blocked, empty, no room) would need edits in both handlers
R4	**Mixed responsibilities**	Data (l.3–13), rules (l.15–52) and DOM (l.55–91) in one file	Rules cannot be tested without guarding for `document`; UI cannot be tested at all
R5	**Mutable booking data**	`room.bookings` is a plain array (l.9)	AC5 holds only by convention; any future code could `push` into it

TASK 01 · CODEBASE ANALYSIS

Prioritised plan (agreed before editing) and a rejected suggestion

P	Step	Fixes
1	Freeze baseline in Git + AC regression tests	safety net
2	Extract `time.js` (parse, overlap)	R2
3	`availability.js`: room passed explicitly, no default	R1, R4
4	`rooms.js` catalog, deep-frozen	R1, R5
5	`filters.js` for F1–F3	new
6	`render.js` + `app.js` controller	R3, R4
7	Run all tests + review diff after each step	—

Rejected: "hide blocked rooms from the list"

The simplest way to satisfy F3 was to filter blocked rooms out entirely. Rejected: a user looking for the Main Auditorium would not know *why* it is missing. Instead the room stays visible with a **BLOCKED** label, and F3 is enforced where it matters — `checkAvailability()` returns `reason: "blocked"`. Covered by test *AC3/F3*.

Modified: input handler in `ui/app.js`

The first generated handler re-validated the slot before deciding whether to refresh — a second copy of AC1 logic, re-creating risk R2. On diff review it was reduced to a single `refresh()` call; all 25 tests still pass.

TASK 02 · FEATURE EXTENSION

Room catalog and filtering rules

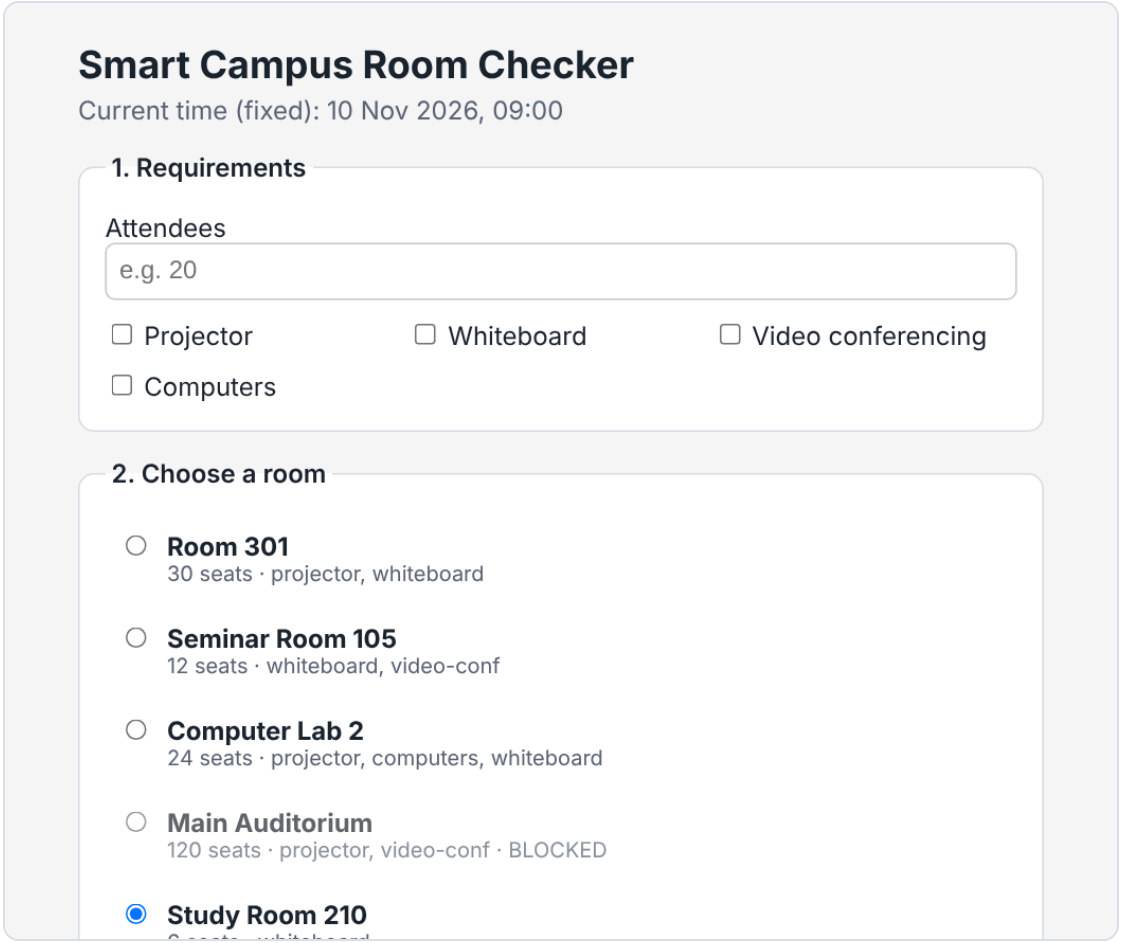
ID	Name	Cap.	Equipment	Blocked	Bookings (10 Nov)
R301	Room 301	30	projector, whiteboard	no	10:00–11:00, 13:00–14:30
R105	Seminar Room 105	12	whiteboard, video-conf	no	11:00–12:00
LAB2	Computer Lab 2	24	projector, computers, whiteboard	no	09:00–10:30
AUD1	Main Auditorium	120	projector, video-conf	**yes**	—
R210	Study Room 210	6	whiteboard	no	—

All data fictional, defined in `final/src/data/rooms.js`, deep-frozen with `Object.freeze`.

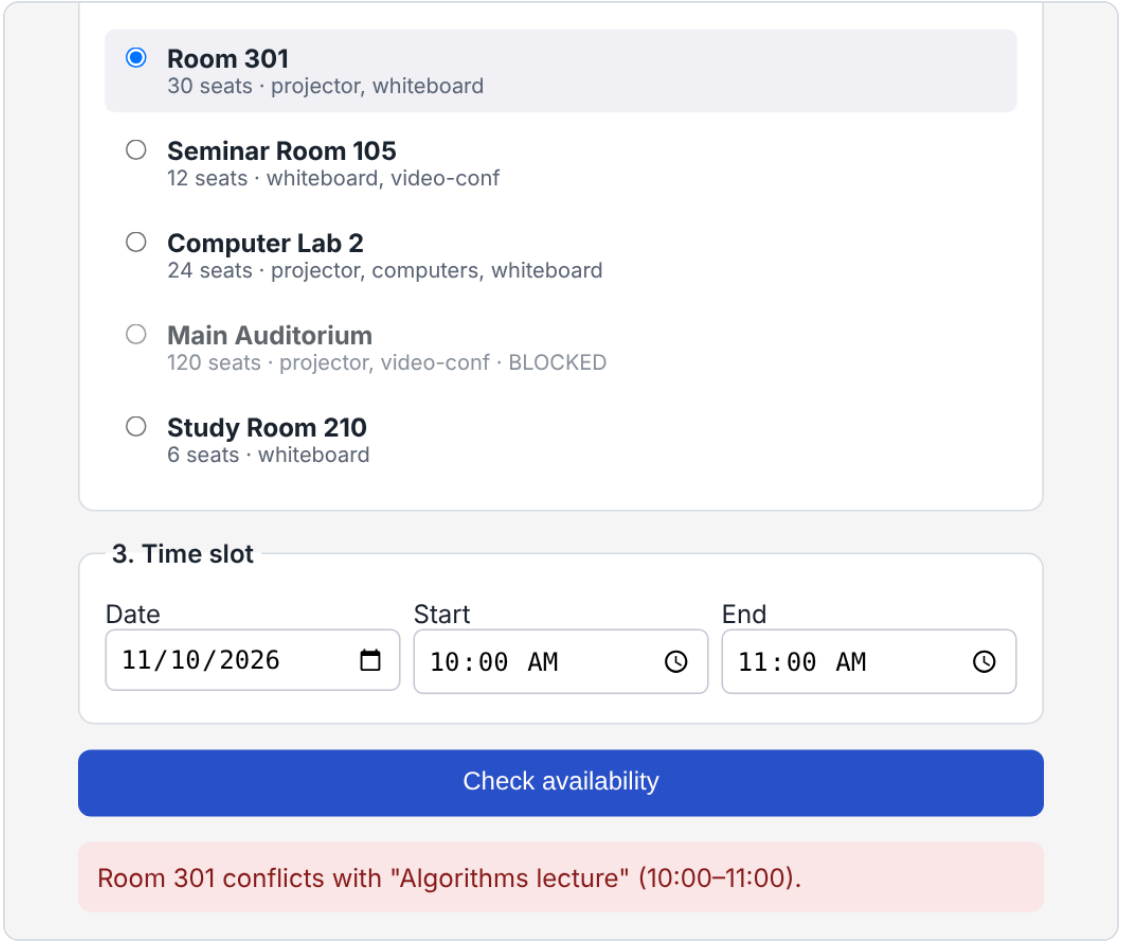
- **F1** `meetsCapacity()`: `capacity >= attendees`; empty input = no requirement
- **F2** `hasAllEquipment()`: `required.every(...)`
- **F3** blocked rooms listed but flagged; `checkAvailability()` never returns ok for them
- **F4** "No rooms match these requirements..." message; selection and result cleared
- **F5** every room change calls `refresh()`, which re-checks with *that* room's bookings via `findRoom()`
- AC1–AC4 logic unchanged in meaning, now in `validateSlot()` + `checkAvailability(room, slot, now)`

TASK 02 · DEMONSTRATION

Successful check, then switching room (F5)



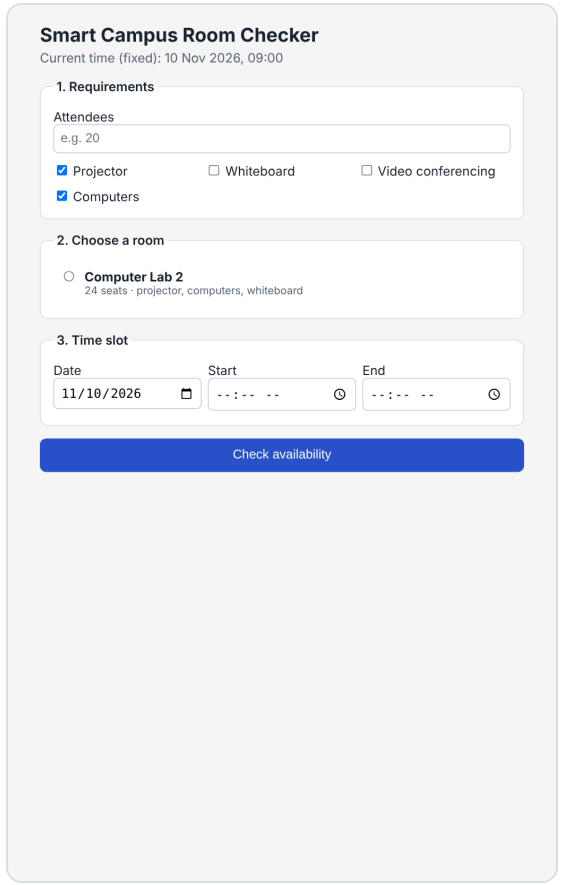
Study Room 210, 10:00–11:00 → available (green).



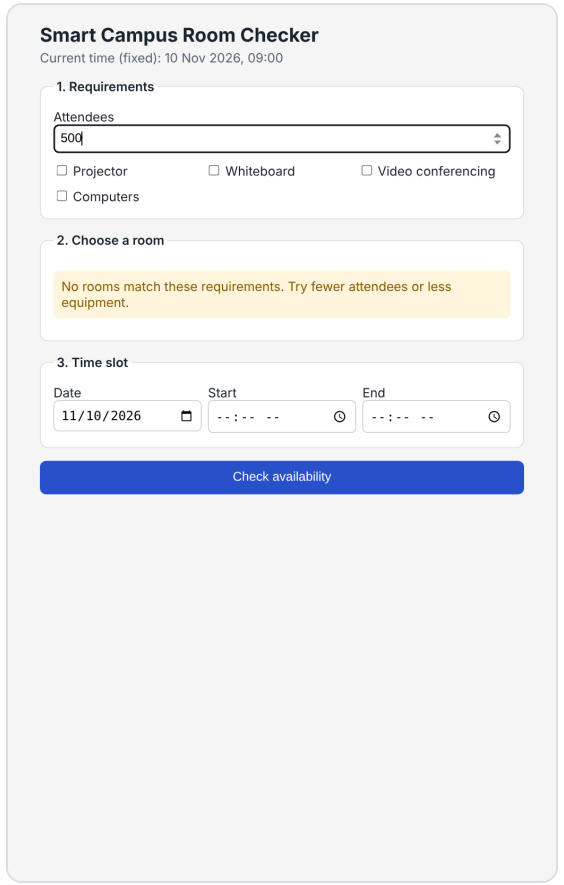
Same slot, switched to Room 301 → result refreshes automatically: conflict with "Algorithms lecture".

TASK 02 · DEMONSTRATION

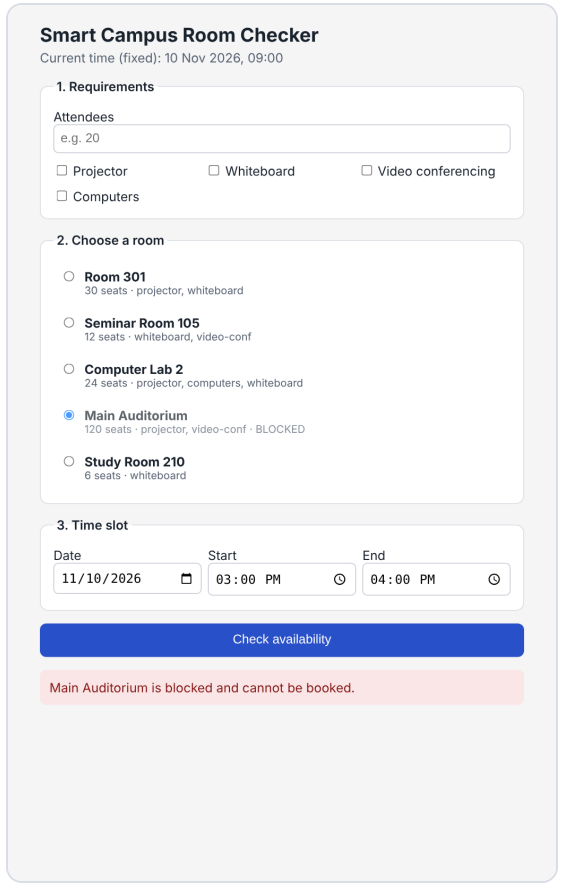
Filtering, empty state, blocked room, 121 minutes on mobile



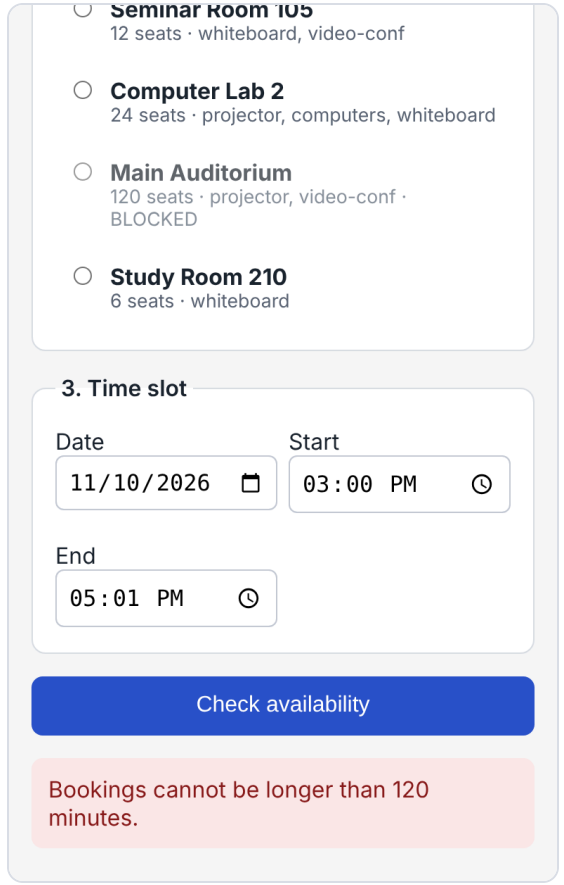
F2: projector + computers → only Computer Lab 2.



F4: 500 attendees → clear empty-state message.



F3: Main Auditorium, free slot → still "blocked".

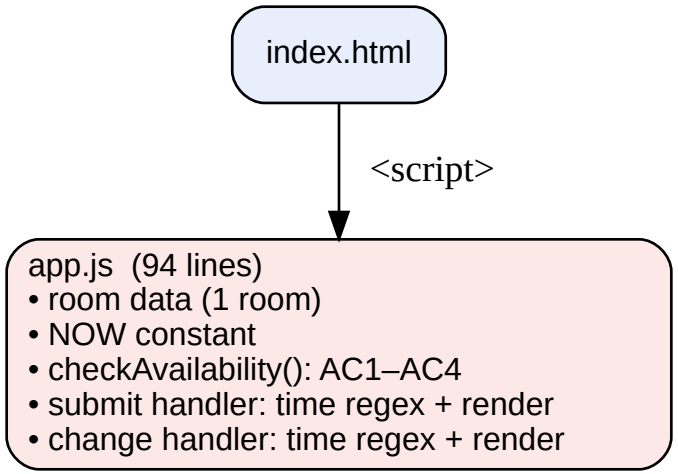


AC2/AC6: 15:00–17:01 at 390 px width → error, layout reflows.

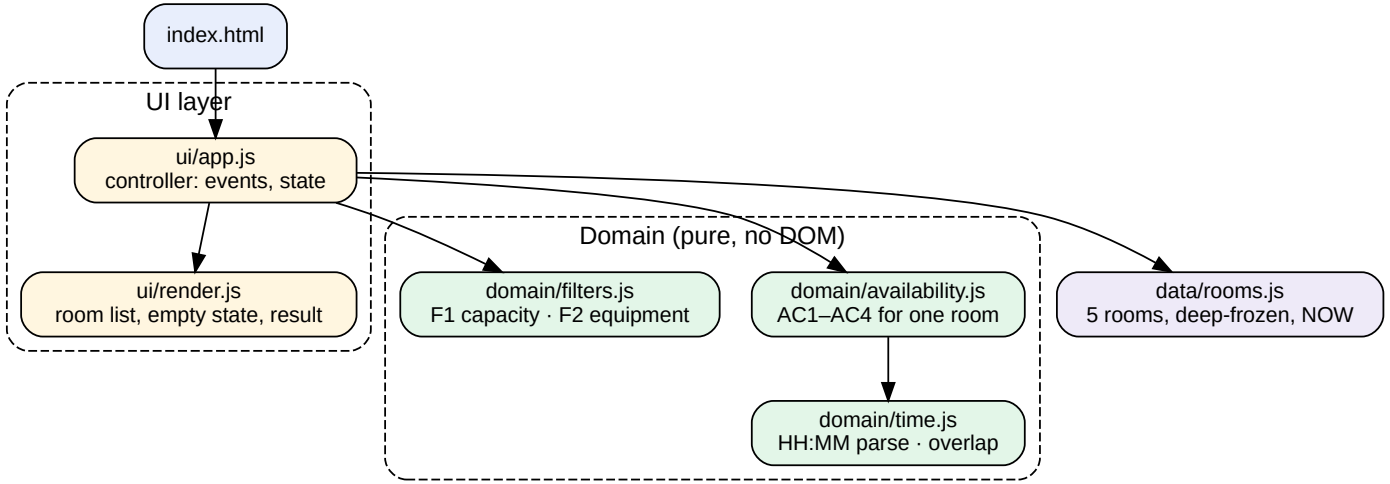
TASK 03 · REFACTORING

Before / after module diagram

Before — 1 module



After — 6 modules in 3 layers



TASK 03 · REFACTORING

Design decisions and their justification

D1. Pure domain layer, room passed explicitly

`checkAvailability(room, slot, now)` has no default room and no DOM access. The baseline fallback `r = r || room` (R1) is gone, so it is impossible to check a slot against another room's bookings by accident. Purity also lets 20 of the 25 tests run in under 2 ms each without a browser.

D2. Single time module with half-open intervals

`time.js` owns HH:MM parsing and `overlaps(a,b) = aStart < bEnd && bStart < aEnd`. One definition replaces three regex copies (R2) and makes the touching-vs-overlap rule explicit. A mutation to `<=` is caught by the AC4 boundary test.

D3. Deep-frozen catalog

AC5 is now enforced by the runtime instead of convention (R5): any write to a booking throws `TypeError` in strict mode, asserted in the AC5 test.

D4. One render path, controller owns state

`render.js` has one `renderResult()` (R3). `app.js` keeps `selectedId`; when filters hide the selected room it is cleared, so a stale result is never shown (F4/F5).

TASK 04 · AUTOMATED TESTING

25 executable tests — Node built-in test runner + jsdom

File	Tests	Covers
`availability.test.js`	12	AC1 (malformed, end<start, past) · **AC2 120 vs 121 min** · AC3/F3 blocked · **AC4 touching 11:00 vs overlapping 10:59**, containing · AC5 frozen & unchanged · same slot busy in R301 / free in R210 · no room selected
`filters.test.js`	8	catalog shape (≥4 rooms, all fields) · **F1 exact capacity 12 vs 13** · empty attendees · F2 ALL items (2 cases) · F1+F2 combined · F4 empty list
`ui.test.js`	5	**Integration/UI** on real `index.html` in jsdom: success feedback · **F5 room switch refresh** · F4 empty state · F1/F2 list update · AC6 error text + viewport
`baseline.test.js`	8	AC1–AC5 against the original `baseline/app.js` (regression reference)

Commands actually run (8 Nov 2026, Node v22.23.4)

```
$ npm install                 # jsdom@24 dev dependency
$ npm run test:baseline       # node --test --test-reporter=spec baseline/tests/*.test.js
$ npm test                    # node --test --test-reporter=spec final/tests/*.test.js
```

Results and checking that the tests can fail

```
✔ AC2 boundary: exactly 120 minutes is allowed
✔ AC2 boundary: 121 minutes is rejected
✔ AC4 boundary: touching interval 11:00–12:00 … allowed
✔ AC4 boundary: overlapping interval 10:59–11:30 rejected
✔ F1 boundary: exact capacity (12) keeps Seminar Room 105
✔ F1 boundary: insufficient capacity (13) removes …
✔ F5 (UI): switching room refreshes result …
✔ F4 (UI): clear empty state when no rooms match …
  … (25 lines total, see docs/evidence/)
ℹ tests 25
ℹ pass 25
ℹ fail 0
```

Baseline suite: **8 / 8** · Final suite: **25 / 25**

Mutation checks (deliberate bugs, then reverted)

Injected bug	Result
`overlaps()`: `<` → `<=` (touching = overlap)	**24 pass / 1 fail** — AC4 touching test
`hasAllEquipment()`: `every` → `some`	**16 pass / 9 fail** — F1, F2 and UI tests

Both bugs were detected, so the suite checks behaviour rather than just running code. Output saved in `docs/evidence/mutation1.txt`, `mutation2.txt`.

Problem found while testing

`node --test baseline/tests/` failed on Node 22 ("Cannot find module") — a directory is not accepted. Fixed with a file glob in `package.json`.

TASK 05 · QUALITY EVALUATION

Original vs final — evidence table

Measure	Original (baseline)	Final	How it was obtained
Automated tests passed / total	8 / 8 (AC1–AC5 only)	25 / 25 (AC1–AC6, F1–F5, UI)	`npm run test:baseline`, `npm test`; summary lines in `docs/evidence/*-output.txt`
Duplicated logic locations removed	HH:MM check ×3, result render ×2	×1 and ×1 → **3 removed**	`grep -c '\d{2}:\d{2}' baseline/app.js` = 3; `grep -c 'out.className = res.ok'` = 2; final `grep -rc 'HHMM.test|className = `result'` = 1 each
Code structure	1 module (94 lines): data + rules + DOM	6 modules (194 lines): data · time · availability · filters · render · controller	`wc -l`; responsibilities from the module diagram (slide 9)
UI behaviour testable?	No (handlers inside `DOMContentLoaded`)	Yes — `mountApp(document)`, 5 jsdom tests	Presence of executable UI tests
Remaining limitations	• Single fixed date/time; bookings cannot be created, only checked · • Equipment names are free strings — no validation that a room's list uses the `EQUIPMENT` vocabulary · • Times use the browser's local time zone · • UI tests use jsdom, not a real browser (layout only checked manually via screenshots) · • No overnight (cross-midnight) slots		

Line counts are a size indicator, not a quality score: the final code is larger because it implements five new requirements.

TASK 06 · AI REFLECTION

Reflection on working with the AI assistant

Context. I gave Claude the full Assignment #04 brief, the submission instructions and the baseline single-room code, with the acceptance criteria AC1–AC6, the new rules F1–F5 and the constraints: fictional data only, no login, APIs or database, and no full rewrite. Stating the boundary values explicitly (120/121 minutes, touching vs overlapping, exact vs insufficient capacity) produced tests aimed at the risky edges instead of generic happy paths.

Useful and incorrect suggestions. The most useful output was the risk list, especially the hidden default room in `checkAvailability()`, which would have let the multi-room version check the wrong room's bookings. Not everything was right: the first input handler duplicated slot validation, recreating the problem the refactor was meant to remove, and the first test script used a directory path that Node 22 rejects. I also rejected hiding blocked rooms, because users would not understand why a room disappeared.

What I verified. I ran both test suites and read the output, injected two deliberate bugs to confirm the tests fail, opened the app and checked each scenario on the screenshots, and recounted the duplication metrics with grep instead of trusting estimates.

Influence of tests. Writing the baseline regression suite before refactoring meant every structural change could be accepted or rejected on evidence. The mutation checks showed the AC4 boundary test is the only guard for touching intervals, so I kept it as a named boundary case.

Next. I would add real-browser end-to-end tests, validate equipment names against one vocabulary, and support creating bookings with immutable updates.

AI-USE RECORD

AI evidence appendix

Tool / date	Prompt summary	Output used	Problems found	Changes made	Verification
Claude Opus 5.5 (Anthropic), claude.ai · 8 Nov 2026	Assignment brief + instructions + baseline code; asked for risk analysis and a small step-by-step plan	Risks R1–R5, 7-step plan (slides 4–5)	None blocking; plan initially hid blocked rooms	Kept blocked rooms visible, flagged	Line references checked against `baseline/app.js`
same session	Implement catalog + F1–F5 one module at a time, keep AC1–AC6	`rooms.js, time.js, availability.js, filters.js, render.js, app.js`	Input handler duplicated AC1 validation	Simplified to single `refresh()`	25/25 tests; diff review; screenshots
same session	Write ≥10 tests incl. listed boundaries + one UI check	3 test files, 25 tests; baseline suite 8 tests	Test script path failed on Node 22	Glob in `package.json`	Commands re-run, output saved
same session	Confirm tests detect real defects	Two mutation runs	—	Mutations reverted	24/25 and 16/25 when mutated; 25/25 after revert
same session	Draft README and this report	README, slides, diagrams (Graphviz)	—	Reviewed and edited	Metrics recomputed with `grep` / `wc`

Verification sources: test runner output (`docs/evidence/`), Git history (`48bd593 → a78e2e9`), headless-Chrome screenshots, Node.js docs for `node --test` and `Object.freeze` .

SUBMISSION

Deliverables and checklist

```
smart-campus/
├─ README.md            setup · run · test
├─ package.json         npm test / test:baseline
├─ baseline/            initial version + 8 tests
├─ final/
│  ├─ index.html
│  ├─ src/data|domain|ui
│  └─ tests/            25 tests
└─ docs/
   ├─ report.pdf        this report
   ├─ img/              screenshots, diagrams
   └─ evidence/         test outputs, mutations, git log
```

Git: `48bd593` baseline → `a78e2e9` final.

Checklist item	Status
Existing availability rules still work	✔ **AC1–AC6 tests**
At least four fictional rooms	✔ **5 rooms**
Capacity and equipment filters work	✔ **F1/F2 tests**
At least ten executable tests	✔ **25 (+8 baseline)**
Refactoring and evaluation evidence	✔ **slides 9–13**
AI use and human review documented	✔ **slides 5, 14, 15**